

Experiment 5: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Theory:

1. Introduction to Support Vector Machine (SVM)

Support Vector Machine (SVM) is a **supervised machine learning algorithm** used for **classification** and **regression** tasks. It is mainly used for **classification problems** and works by finding an **optimal hyperplane** that separates data points of different classes with **maximum margin**.

SVM is highly effective for **high-dimensional datasets** and performs well even when the **number of features** is large compared to the **number of samples**.

2. Dataset Used

For this experiment, the **Telco Customer Churn Dataset** was used.

Dataset Name:

Telco Customer Churn Dataset

This dataset contains customer information from a telecom company and aims to predict whether a customer will churn (leave the service) or not.

Dataset Description:

- Total Records: 7043
- Features: 20+ customer-related features
- Target Variable: Churn (Yes / No)
- Type: Binary Classification Problem

Important Features:

- Tenure
- Monthly Charges
- Total Charges
- Contract Type
- Internet Service
- Payment Method

Target Variable:

- 1 → Customer Churned
- 0 → Customer Did Not Churn

3. Mathematical Formulation of SVM

The goal of SVM is to find a hyperplane:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- x = feature vector
- b = bias

For correctly classified points:

$$y_i(w \cdot x_i + b) \geq 1$$

SVM tries to maximize the margin:

$$\text{Margin} = \frac{2}{||w||}$$

This is achieved by solving the optimization problem:

$$\min \frac{1}{2} ||w||^2$$

For non-linearly separable data, SVM introduces:

- Slack variable (ξ)
- Regularization parameter (C)

Modified objective:

$$\min \frac{1}{2} ||w||^2 + C \sum \xi_i$$

4. Kernel Trick

If data is not linearly separable, SVM uses kernel functions to transform data into higher-dimensional space.

Common Kernels:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel

RBF kernel formula:

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

Where:

- γ controls the influence of a single training example.

5. Hyperparameters in SVM

Important hyperparameters:

- **C**: Regularization parameter
 - Large C → Less margin, more accuracy on training data
 - Small C → Larger margin, more generalization
- **Kernel**: Type of decision boundary
- **Gamma** (for RBF kernel):
 - High gamma → Overfitting
 - Low gamma → Underfitting

Hyperparameter tuning was performed using GridSearchCV to find the best values of C, gamma, and kernel.

6. Methodology / Workflow

The experiment followed these steps:

1. Load dataset
2. Handle missing values
3. Convert categorical variables

4. Feature scaling using StandardScaler
5. Split dataset into training and testing sets
6. Train SVM model
7. Perform hyperparameter tuning
8. Evaluate using:
 - Accuracy
 - Confusion Matrix
 - Classification Report
9. Visualize results

7. Performance Evaluation Metrics

The model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Accuracy Formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

8. Limitations of SVM

- Not suitable for very large datasets (slow training time)
- Sensitive to choice of kernel and hyperparameters
- Requires feature scaling
- Less interpretable compared to Decision Trees

Code:

```
# Import Required Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

import seaborn as sns
import matplotlib.pyplot as plt

# Load Dataset

# Read CSV file (Make sure file is uploaded in Colab)
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

# Display first 5 rows
print(df.head())

# Data Cleaning

# Drop customerID column (not useful for prediction)
if "customerID" in df.columns:
    df = df.drop("customerID", axis=1)

# Convert TotalCharges column to numeric
# errors='coerce' converts invalid values to NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Fill missing values with median
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
```

```
# Encode Categorical Variables

# Initialize LabelEncoder
le = LabelEncoder()

# Convert all object (categorical) columns into numeric
for column in df.columns:
    if df[column].dtype == "object":
        df[column] = le.fit_transform(df[column])

# Define Features and Target

# X → Independent variables
X = df.drop("Churn", axis=1)

# y → Target variable
y = df["Churn"]

# Split Dataset into Training and Testing Sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling (Important for SVM)

scaler = StandardScaler()

# Fit scaler on training data and transform
X_train = scaler.fit_transform(X_train)

# Transform test data using same scaler
X_test = scaler.transform(X_test)

# Hyperparameter Tuning using GridSearchCV

# Define parameter grid
param_grid = {
    'C': [0.1, 1, 10],          # Regularization parameter
    'kernel': ['linear', 'rbf', 'poly'], # Different kernel types
    'gamma': ['scale', 'auto']    # Kernel coefficient
}
```

```
# Create SVM model
svm = SVC()

# Apply GridSearch with 5-fold cross validation
grid = GridSearchCV(svm, param_grid, cv=5, verbose=2)

# Train model with different hyperparameter combinations
grid.fit(X_train, y_train)

# Print best parameters
print("Best Parameters:", grid.best_params_)

# Evaluate Best Model

# Get best model from GridSearch
best_model = grid.best_estimator_

# Predict on test data
y_pred = best_model.predict(X_test)

# Print evaluation metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Visualize Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# Compare Different Kernels

kernels = ['linear', 'rbf', 'poly']
accuracies = []

# Train and evaluate SVM with different kernels
for k in kernels:
    model = SVC(kernel=k)
```



```

model.fit(X_train, y_train)
pred = model.predict(X_test)
accuracies.append(accuracy_score(y_test, pred))

```

```

# Plot accuracy comparison
plt.figure()
plt.plot(kernels, accuracies)
plt.xlabel("Kernel")
plt.ylabel("Accuracy")
plt.title("Accuracy for Different Kernels")
plt.show()

```

Output:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
2	3668-QFYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes

5 rows × 21 columns

Fitting 5 folds for each of 18 candidates, totalling 90 fits

```

[CV] END .....C=0.1, gamma=scale, kernel=linear; total time= 0.9s
[CV] END .....C=0.1, gamma=scale, kernel=linear; total time= 0.9s
[CV] END .....C=0.1, gamma=scale, kernel=linear; total time= 1.0s
[CV] END .....C=0.1, gamma=scale, kernel=linear; total time= 1.8s
[CV] END .....C=0.1, gamma=scale, kernel=linear; total time= 1.1s
[CV] END .....C=0.1, gamma=scale, kernel=rbf; total time= 1.2s
[CV] END .....C=0.1, gamma=scale, kernel=rbf; total time= 0.8s
[CV] END .....C=0.1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=scale, kernel=poly; total time= 0.6s
[CV] END .....C=0.1, gamma=scale, kernel=poly; total time= 0.6s
[CV] END .....C=0.1, gamma=scale, kernel=poly; total time= 0.5s
[CV] END .....C=0.1, gamma=scale, kernel=poly; total time= 0.5s
[CV] END .....C=0.1, gamma=scale, kernel=poly; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=linear; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=linear; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=linear; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=linear; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=linear; total time= 0.5s
[CV] END .....C=0.1, gamma=auto, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=auto, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=auto, kernel=rbf; total time= 0.7s
[CV] END .....C=0.1, gamma=auto, kernel=rbf; total time= 1.9s
[CV] END .....C=0.1, gamma=auto, kernel=rbf; total time= 2.1s
[CV] END .....C=0.1, gamma=auto, kernel=poly; total time= 1.0s
[CV] END .....C=0.1, gamma=auto, kernel=poly; total time= 1.6s
[CV] END .....C=0.1, gamma=auto, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, gamma=auto, kernel=poly; total time= 1.9s
[CV] END .....C=0.1, gamma=auto, kernel=poly; total time= 2.2s
[CV] END .....C=1, gamma=scale, kernel=linear; total time= 4.3s
[CV] END .....C=1, gamma=scale, kernel=linear; total time= 1.9s
[CV] END .....C=1, gamma=scale, kernel=linear; total time= 1.9s
[CV] END .....C=1, gamma=scale, kernel=linear; total time= 1.9s
[CV] END .....C=1, gamma=scale, kernel=linear; total time= 1.3s
[CV] END .....C=1, gamma=scale, kernel=rbf; total time= 0.8s
[CV] END .....C=1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=1, gamma=scale, kernel=rbf; total time= 0.7s
[CV] END .....C=1, gamma=scale, kernel=rbf; total time= 1.1s
[CV] END .....C=1, gamma=scale, kernel=poly; total time= 1.0s
[CV] END .....C=1, gamma=scale, kernel=poly; total time= 1.0s
[CV] END .....C=1, gamma=scale, kernel=poly; total time= 0.7s
[CV] END .....C=1, gamma=scale, kernel=poly; total time= 0.6s
[CV] END .....C=1, gamma=scale, kernel=poly; total time= 0.6s
[CV] END .....C=1, gamma=auto, kernel=linear; total time= 0.9s
[CV] END .....C=1, gamma=auto, kernel=linear; total time= 0.9s
[CV] END .....C=1, gamma=auto, kernel=linear; total time= 0.9s
[CV] END .....C=1, gamma=auto, kernel=linear; total time= 0.9s

```

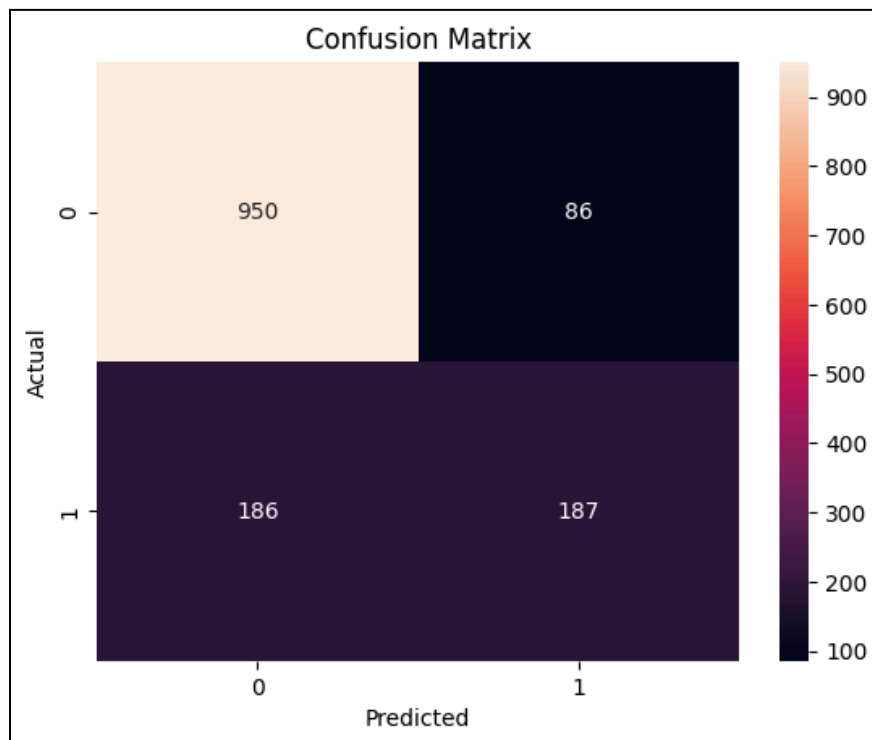
Accuracy: 0.8069552874378992

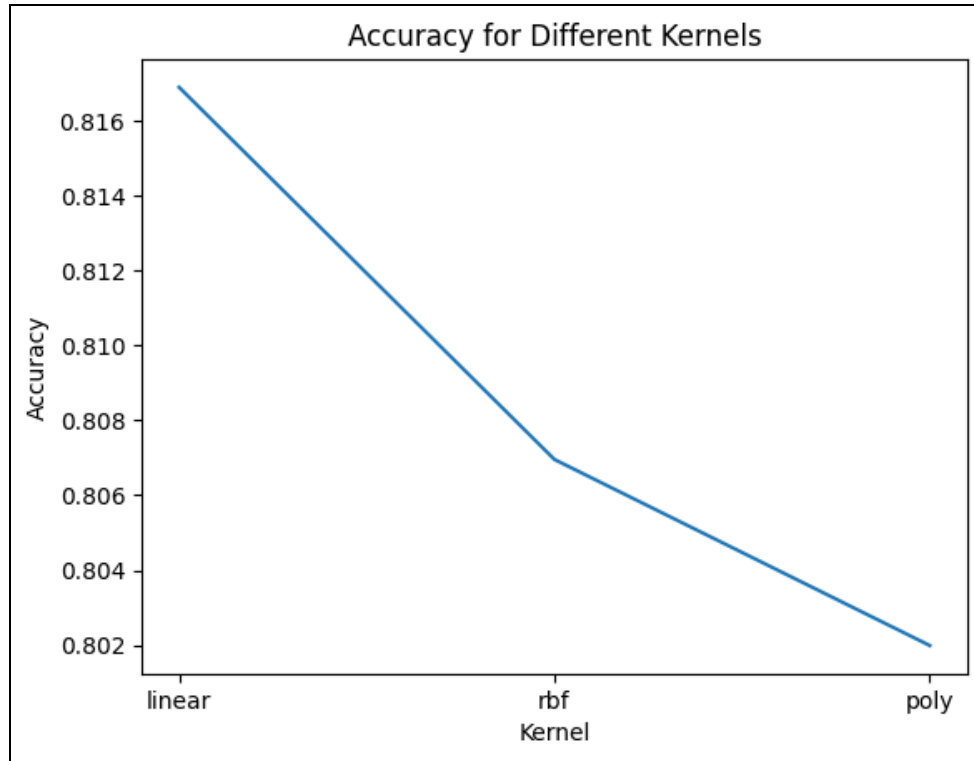
Classification Report:

	precision	recall	f1-score	support
0	0.84	0.92	0.87	1036
1	0.68	0.50	0.58	373
accuracy			0.81	1409
macro avg	0.76	0.71	0.73	1409
weighted avg	0.80	0.81	0.80	1409

Confusion Matrix:

```
[[950  86]
 [186 187]]
```





Conclusion:

The **Support Vector Machine (SVM)** algorithm was successfully applied to **customer churn classification**. After comprehensive **data preprocessing**, **feature scaling**, and **hyperparameter tuning** via **GridSearchCV**, the optimized model achieved an overall **accuracy of 0.81**. The best parameters identified were **C = 10**, **gamma = 'scale'**, and **kernel = 'rbf'**. Notably, a simpler **linear kernel** demonstrated a slightly higher **accuracy of 0.817** in separate tests, suggesting that while **hyperparameter tuning** is valuable, **kernel selection** can significantly impact performance on structured datasets. Despite potential **computational costs**, SVM proved to be a **robust classifier** for this prediction task.